

2주차(네트워크 보안)

≡ 멘토	이효민 멘토님
≡ 과제	O

PCAP Programming 과제 보고서

깃허브 링크 : <https://github.com/tortoise/WHS-4TH-study/tree/main/2%EC%A3%BC%EC%B0%A8/%EB%84%A4%ED%8A%B8%EC%9B%8C%ED%81%AC%EB%B3%B4>

C 기반 패킷 스니퍼(Packet Sniffer) 구현 및 분석

| 화이트햇스쿨(White Hat School) 4기 | 4반 | 이승환(8419)

1. 과제 개요

이번 과제에서는 PCAP API를 활용하여 네트워크를 오가는 **패킷(Packet)** 을 직접 캡처하고, 그 안에 담긴 정보를 계층별로 출력하는 프로그램을 C 언어와 **libpcap** 라이브러리로 작성하였다. 쉽게 말해 Wireshark 같은 패킷 분석 도구가 하는 일을 작은 규모로 직접 구현해 보는 실습이었다.

출력해야 하는 정보는 다음 네 가지 계층으로 나뉜다.

- **Ethernet Header** — 출발지 MAC 주소 / 목적지 MAC 주소
- **IP Header** — 출발지 IP / 목적지 IP
- **TCP Header** — 출발지 포트 / 목적지 포트
- **HTTP Message** — 애플리케이션 계층의 실제 데이터(payload)

단, **TCP 패킷만 처리** 하고 UDP는 무시하는 것이 조건이었다. 또한 IP 헤더와 TCP 헤더에 들어 있는 **길이 정보**(`ip_header_len` 등)를 반드시 활용해야 했는데, 이 부분에서 프로토콜 구조를 깊이 이해할 수 있었다.

2. 패킷 캡처는 왜 하는가

이번 실습에서는 단순히 코드를 작성하는 데 그치지 않고, 패킷 캡처가 실제로 어떤 의미를 갖는지 함께 이해하고자 하였다. 네트워크로 데이터를 주고받을 때 모든 정보는 패킷이라는 작은 조각으로 나뉘어 오가는데, 패킷 캡처란 이 오가는 조각들을 중간에서 복사하여 살펴보는 작업이다.

이는 마치 우체국에서 오가는 편지를 복사하여 누가 누구에게 무엇을 보냈는지 확인하는 것과 유사하다고 이해하였다. 실제로 이번 과제에서 출력한 정보만 보아도, 패킷 하나로 다음과 같은 내용을 모두 확인할 수 있었다.

- **MAC 주소** → 어느 기계(랜카드)에서 나갔는지
- **IP 주소** → 어느 컴퓨터에서 어느 컴퓨터로 갔는지 (편지의 발신·수신 주소)
- **포트 번호** → 그 컴퓨터의 어느 프로그램으로 갔는지 (80=웹, 443=HTTPS 등)
- **HTTP 메시지** → 실제로 주고받은 데이터 (`GET / HTTP/1.1` = "웹페이지를 달라"는 요청)

즉 패킷 하나만 살펴봐도 "누가, 누구에게, 무엇으로, 무엇을" 보냈는지 모두 드러난다는 점을 알 수 있었다. 이러한 특성은 방어에도 공격에도 쓰이는 양날의 검이라는 것을 이해하는 계기가 되었다.

방어 사례 — 해커 침입 탐지 (IDS)

회사 네트워크 입구에서 오가는 패킷을 계속 감시하는 상황을 생각해 보았다. 만약 다음과 같은 패킷이 잡힌다면,

```
Dst Port: 3306 (데이터베이스 포트)
HTTP Message: ' OR '1'='1 ...
```

이는 평소에는 없던 **SQL Injection 공격 문자열** 이 데이터베이스로 날아가고 있는 상황이다. 감시 시스템은 이를 공격으로 판단하여 경보를 울리고 해당 IP를 차단하게 된다. 이것이 바로 **침입 탐지 시스템(IDS)** 의 원리이며, Snort·Suricata 같은 실제 보안 장비가 이 방식으로 동작한다는 것을 알 수 있었다.

공격 사례 — 로그인 정보 탈취 (HTTPS가 필요한 이유)

반대로 카페 공용 와이파이에서 패킷 캡처를 켜 둔 상황도 생각해 보았다. 옆 사람이 **암호화되지 않은 HTTP 사이트** 에 로그인하면 다음과 같은 패킷이 잡힌다.

```
Dst Port: 80
HTTP Message: POST /login ... id=user123&password=mypass1234
```

이처럼 **아이디와 비밀번호가 평문 그대로 노출** 되는데, 이것이 과거 공용 와이파이가 위험했던 진짜 이유였음을 이해하게 되었다. 그래서 오늘날 거의 모든 사이트가 **HTTPS(443번, 암호화)** 를 사용하며, HTTPS를 쓰면 패킷을 캡처해도 내용이 암호문이라 읽을 수 없다. 이번 실습에서 `curl http://...` (https가 아닌 http)를 사용하는 이유도 바로 이 때문이라는 것을 알 수 있었다.

이번 과제를 통해 오가는 데이터는 기본적으로 누구나 들여다볼 수 있다는 점, 그렇기 때문에 **암호화(HTTPS/TLS)** 가 왜 필수인지를 근본적으로 이해할 수 있었다.

3. 사전 지식 — 패킷은 "양파" 구조다

이 과제를 이해하는 데 가장 중요했던 개념은 **패킷이 여러 겹의 헤더로 겹겹이 싸여 있다** 는 사실이었다.

이는 택배를 부치는 과정에 비유하여 이해하였다. 편지(실제 데이터)를 봉투에 넣고(TCP), 그 봉투를 다시 택배 상자에 넣어 주소를 적고(IP), 그 상자를 트럭에 실어 배송 라벨을 붙이는 것(Ethernet)과 같다. 받는 사람은 이 과정을 거꾸로 하나씩 풀면서 안쪽으로 들어가 결국 편지를 꺼내게 되는데, 패킷 분석도 정확히 이 "겉포장부터 하나씩 벗기는" 과정임을 알 수 있었다.

메모리 상에서 패킷은 이 포장들이 앞에서부터 순서대로 붙어 있는 하나의 연속된 바이트 덩어리다.

```
[ Ethernet 헤더 ][ IP 헤더 ][ TCP 헤더 ][ 실제 데이터 (HTTP Message) ]
14 bytes 고정   20 bytes~   20 bytes~   나머지 전부
              (가변)       (가변)
```

따라서 프로그래밍의 핵심 원리는 "지금 위치 + 이 헤더의 크기 = 다음 헤더의 시작 위치"로 요약할 수 있었다. 이렇게 포인터를 헤더 크기 만큼 계속 이동시키면서 한 겹씩 안쪽으로 파고드는 방식으로 구현하였다.

4. 왜 "길이 정보"가 반드시 필요한가

Ethernet 헤더는 항상 **14바이트로 고정** 이기 때문에, IP 헤더의 시작 위치는 언제나 "맨 앞 + 14바이트"로 간단히 구할 수 있었다. 문제는 그 다음이었다.

IP 헤더와 TCP 헤더는 크기가 변할 수 있다. 기본은 각각 20바이트지만, '옵션(option)' 필드가 붙으면 더 커진다. 만약 코드에서 "IP 헤더는 20바이트겠지"라고 고정값을 넣으면, 옵션이 붙은 패킷을 만났을 때 TCP 헤더의 시작 위치를 잘못 계산하게 되고, 결국 엉뚱한 메모리를 읽어 틀린 결과가 나온다는 것을 알 수 있었다.

그래서 헤더 안에 들어 있는 **길이 필드** 를 직접 읽어서 실제 크기를 계산해야 했다.

구하려는 값	계산식	이유
IP 헤더 길이	<code>iph_ihl × 4</code>	IHL 필드는 바이트가 아니라 '4바이트 묶음(word)' 개수로 저장됨
TCP 헤더 길이	<code>data_offset × 4</code>	TCP의 offset 필드도 동일하게 word 단위

여기서 왜 `× 4` 를 하는지도 이해할 수 있었다. 이 길이 필드들은 "몇 바이트"가 아니라 "4바이트짜리 묶음이 몇 개"인지를 저장한다. 예를 들어 `iph_ihl` 값이 5라면, 실제 길이는 $5 \times 4 = 20$ 바이트 가 된다. 계란을 "30개" 대신 "한 판(=30개)"으로 세는 것과 같은 방식이라고 이해하니 명확해졌다.

HTTP 메시지(실제 데이터)의 길이 구하기

IP 헤더 안에는 `iph_len` 이라는 필드가 있는데, 이는 IP 헤더부터 패킷 끝까지의 전체 길이를 담고 있다. 여기서 두 헤더의 크기를 빼면 순수한 데이터(HTTP 메시지)의 길이가 남는다는 것을 알 수 있었다.

HTTP 데이터 길이 = `iph_len - (IP 헤더 길이 + TCP 헤더 길이)`

이 값이 0보다 크면 실제 데이터가 존재한다는 뜻이고, 그 부분을 문자로 출력하면 `GET / HTTP/1.1` 같은 HTTP 메시지를 확인할 수 있었다.

5. 전체 동작 흐름

프로그램은 다음 순서로 동작하도록 작성하였다.

1. 네트워크 카드(NIC)를 열어 패킷을 잡을 준비를 한다. (`pcap_open_live`)
2. "TCP만 잡아라"라는 필터를 설정한다. (`pcap_setfilter`)
3. 패킷이 하나 잡힐 때마다 `got_packet` 함수를 자동 호출하는 무한 루프를 돈다. (`pcap_loop`)
4. 잡힌 패킷에서 Ethernet → IP → TCP → HTTP 순으로 한 겹씩 파고들며 정보를 출력한다.

6. 코드 및 상세 해석

구조체는 "메모리의 이 위치부터 이런 순서로 데이터가 들어 있다"고 알려주는 **틀(template)** 이다. 패킷 메모리 위에 이 구조체 포인터를 씌우면(캐스팅), 각 필드를 이름으로 편하게 꺼낼 수 있다는 점을 활용하였다.

6-1. myheader.h — 헤더 구조체 정의

```
#ifndef MYHEADER_H
#define MYHEADER_H

#include <netinet/in.h>

/* Ethernet 헤더 (14 bytes 고정) */
#define ETHER_ADDR_LEN 6
struct ethheader {
    u_char ether_dhost[ETHER_ADDR_LEN]; /* 목적지 MAC */
    u_char ether_shost[ETHER_ADDR_LEN]; /* 출발지 MAC */
    u_short ether_type; /* 0x0800 = IP */
};

/* IP 헤더 */
struct ipheader {
    unsigned char iph_ihl:4, /* IP 헤더 길이 (4바이트 word 단위) * */
                  iph_ver:4; /* IP 버전 */
    unsigned char iph_tos; /* 서비스 타입 */
    unsigned short int iph_len; /* IP 전체 길이 * */
    unsigned short int iph_ident; /* 식별자 */
    unsigned short int iph_flag:3, /* 플래그 */
                  iph_offset:13; /* 프래그먼트 오프셋 */
    unsigned char iph_ttl; /* TTL */
    unsigned char iph_protocol; /* 프로토콜 (6 = TCP) * */
    unsigned short int iph_chksum; /* 체크섬 */
    struct in_addr iph_sourceip; /* 출발지 IP */
    struct in_addr iph_destip; /* 목적지 IP */
};

/* TCP 헤더 */
struct tcpheader {
    u_short tcp_sport; /* 출발지 포트 */
    u_short tcp_dport; /* 목적지 포트 */
    u_int tcp_seq; /* 시퀀스 번호 */
    u_int tcp_ack; /* ACK 번호 */
    u_char tcp_offx2; /* data offset (상위 4비트) * */
#define TH_OFF(th) (((th)->tcp_offx2 & 0xf0) >> 4)
    u_char tcp_flags; /* 플래그 */
    u_short tcp_win; /* 윈도우 크기 */
    u_short tcp_sum; /* 체크섬 */
};
```

```

    u_short tcp_urp;                                /* 긴급 포인터 */
};

#endif

```

여기서 `iph_ihl:4`의 `:4`는 "이 변수는 1바이트 중 4비트만 쓴다"는 **비트필드(bit-field)** 문법이다. IP 버전(4비트)과 헤더 길이(4비트)가 한 바이트를 절반씩 나눠 쓰기 때문에 이렇게 쪼개서 선언한다는 것을 알 수 있었다.

6-2. sniff_improved.c — 메인 로직

패킷이 잡힐 때마다 실행되는 `got_packet` 함수가 이 프로그램의 핵심이다. 한 겹씩 파고드는 흐름에 주목하여 작성하였다.

```

#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include "myheader.h"

/* MAC 주소를 xx:xx:xx:xx:xx:xx 형식으로 출력 */
void print_mac(const char *label, const u_char *mac) {
    printf("    %s: %02x:%02x:%02x:%02x:%02x:%02x\n",
        label, mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
}

/* 패킷이 하나 잡힐 때마다 자동 호출되는 콜백 함수 */
void got_packet(u_char *args, const struct pcap_pkthdr *header,
    const u_char *packet) {

    /* [1겹] 패킷 맨 앞에 Ethernet 구조체 틀을 씌운다 */
    struct ethheader *eth = (struct ethheader *)packet;

    /* IP 패킷(0x0800)이 아니면 무시. ntohs = 바이트 순서 변환 */
    if (ntohs(eth->ether_type) != 0x0800) return;

    /* [2겹] Ethernet(14바이트)을 건너뛴 위치가 IP 헤더 */
    struct ipheader *ip =
        (struct ipheader *) (packet + sizeof(struct ethheader));

    /* * TCP가 아니면(protocol != 6) 무시 → UDP 등 제외 */
    if (ip->iph_protocol != IPPROTO_TCP) return;

    /* * IP 헤더 실제 길이 = ihl × 4 (고정 20이 아니라 계산해서 사용) */
    int ip_header_len = ip->iph_ihl * 4;

    /* [3겹] IP 헤더 길이만큼 건너뛴 위치가 TCP 헤더 */
    struct tcpheader *tcp =
        (struct tcpheader *) ((u_char *)ip + ip_header_len);

    /* * TCP 헤더 실제 길이 = data offset × 4 */
    int tcp_header_len = TH_OFF(tcp) * 4;

    printf("\n===== TCP Packet =====\n");

    /* --- Ethernet Header --- */
    printf("Ethernet Header\n");
    print_mac("Src MAC", eth->ether_shost);
    print_mac("Dst MAC", eth->ether_dhost);
}

```

```

/* --- IP Header (inet_ntoa: 숫자 IP를 문자열로) --- */
printf("IP Header\n");
printf("   Src IP: %s\n", inet_ntoa(ip->iph_sourceip));
printf("   Dst IP: %s\n", inet_ntoa(ip->iph_destip));

/* --- TCP Header (ntohs: 포트 번호 바이트 순서 변환) --- */
printf("TCP Header\n");
printf("   Src Port: %d\n", ntohs(tcp->tcp_sport));
printf("   Dst Port: %d\n", ntohs(tcp->tcp_dport));

/* [4결] --- HTTP Message (payload) --- */
int total_headers = sizeof(struct ethheader) + ip_header_len + tcp_header_len;
/* * 데이터 길이 = 전체 IP 길이 - (IP 헤더 + TCP 헤더) */
int payload_len = ntohs(ip->iph_len) - (ip_header_len + tcp_header_len);
const u_char *payload = packet + total_headers;

printf("HTTP Message\n");
if (payload_len > 0) {
    int len = payload_len > 100 ? 100 : payload_len; /* 최대 100바이트만 */
    printf("   ");
    for (int i = 0; i < len; i++) {
        /* 눈에 보이는 문자만 출력, 나머지는 점(.)으로 대체 */
        if (payload[i] >= 32 && payload[i] <= 126)
            printf("%c", payload[i]);
        else
            printf(".");
    }
    printf("\n");
} else {
    printf("   (No Payload)\n");
}
}
}

```

6-3. main 함수 — 준비와 실행

```

int main() {
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "tcp"; /* TCP만 캡처하는 필터 */
    bpf_u_int32 net = 0;

    /* NIC 열기. "eth0"은 본인 환경에 맞게 수정 (ens33, enp0s3 등) */
    handle = pcap_open_live("eth0", BUFSIZ, 1, 1000, errbuf);
    if (handle == NULL) {
        fprintf(stderr, "Couldn't open device: %s\n", errbuf);
        return 2;
    }

    /* 필터 컴파일 & 적용 */
    if (pcap_compile(handle, &fp, filter_exp, 0, net) == -1) {
        fprintf(stderr, "Filter compile error\n");
        return 2;
    }
    if (pcap_setfilter(handle, &fp) == -1) {
        fprintf(stderr, "Filter set error\n");
    }
}

```

```

        return 2;
    }

    /* -1 = 무한 반복. 패킷 잡을 때마다 got_packet 호출 */
    pcap_loop(handle, -1, got_packet, NULL);

    pcap_close(handle);
    return 0;
}

```

7. 자주 등장하는 함수 정리

실습 과정에서 반복적으로 사용한 함수들을 정리하였다.

함수 / 개념	역할
<code>pcap_open_live()</code>	지정한 네트워크 카드를 열어 패킷 캡처를 시작할 수 있게 한다.
<code>pcap_loop()</code>	패킷이 잡힐 때마다 콜백 함수(<code>got_packet</code>)를 반복 호출한다.
<code>ntohs()</code>	Network TO Host Short. 네트워크(빅엔디안)의 2바이트 숫자를 내 PC 순서로 변환. 포트·타입 값에 필수.
<code>inet_ntoa()</code>	4바이트 숫자로 된 IP 주소를 <code>192.168.0.1</code> 같은 읽기 쉬운 문자열로 변환.
포인터 캐스팅	연속된 바이트 덩어리(packet) 위에 구조체 틀을 씌워, 각 필드를 이름으로 접근하게 해주는 핵심 기법.

8. 컴파일 및 실행 방법

```

# 1. libpcap 개발 라이브러리 설치
sudo apt-get install libpcap-dev

# 2. 컴파일 (-lpcap 링크 옵션 필수)
gcc sniff_improved.c -o sniff -lpcap

# 3. 실행 (패킷 캡처는 관리자 권한 필요)
sudo ./sniff

```

실행한 뒤, 다른 터미널에서 아래 명령으로 HTTP 트래픽을 발생시키면 패킷이 잡히면서 HTTP 메시지가 출력된다. (HTTPS가 아닌 HTTP여야 내용이 암호화되지 않아 확인할 수 있다.)

```
curl http://example.com
```

실행 결과

- 처음 vs코드 powershell 에서 실행하였을 때는 curl문이 실행되지 않아, 당황하였으나 wsl 을 끌어온뒤 우분투 환경을 설치하고, 스니프 코드를 동작하는데 필요한 라이브러리들을 설치후 실행하니 정상적으로 동작하였다.
- 패킷 캡처를 하기 위해서는 두개의 터미널을 열고 한개의 터미널에서는 \$ sudo ./sniff 를 입력하여 패킷캡처가 가능한 상태로 대기시키고, 다른 한개의 터미널에서는 \$ curl http://example.com 를입력하여 패킷을 전송하여야 한다.

```

samsungkjh@ESKTOP-VGQNDZ:/mnt/d/이승환/영지대학교/화이트햇스쿨/2주차/네트워크보안$ curl http://example.com
<doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">learn more</a></p></div></body></html>

```

```
samsungkjhu@DESKTOP-V6QNDE2:/mnt/d/이승환/명지대학교/화이트햇스쿨$ cd ~/mnt/d/이승환/명지대학교/화이트햇스쿨/2주차/네트워크보안
samsungkjhu@DESKTOP-V6QNDE2:/mnt/d/이승환/명지대학교/화이트햇스쿨/2주차/네트워크보안$ gcc sniff_improved.c -o sniff -lpcap
samsungkjhu@DESKTOP-V6QNDE2:/mnt/d/이승환/명지대학교/화이트햇스쿨/2주차/네트워크보안$ sudo ./sniff
[sudo: authenticate] Password:
0000

0000
curl http://example.com

curl http://example.com

===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:18:e8:6c
  Dst MAC: 00:15:5d:59:d8:3d
IP Header
  Src IP: 172.22.8.145
  Dst IP: 172.66.147.243
TCP Header
  Src Port: 43484
  Dst Port: 80
HTTP Message
  (No Payload)

===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:59:d8:3d
  Dst MAC: 00:15:5d:18:e8:6c
IP Header
  Src IP: 172.66.147.243
  Dst IP: 172.22.8.145
TCP Header
  Src Port: 80
  Dst Port: 43484
HTTP Message
  (No Payload)
```

```
===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:18:e8:6c
  Dst MAC: 00:15:5d:59:d8:3d
IP Header
  Src IP: 172.22.8.145
  Dst IP: 172.66.147.243
TCP Header
  Src Port: 43484
  Dst Port: 80
HTTP Message
  GET / HTTP/1.1..Host: example.com..User-Agent: curl/8.18.0..Accept: /*/*....

===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:59:d8:3d
  Dst MAC: 00:15:5d:18:e8:6c
IP Header
  Src IP: 172.66.147.243
  Dst IP: 172.22.8.145
TCP Header
  Src Port: 80
  Dst Port: 43484
HTTP Message
  (No Payload)

===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:59:d8:3d
  Dst MAC: 00:15:5d:18:e8:6c
IP Header
  Src IP: 172.66.147.243
  Dst IP: 172.22.8.145
TCP Header
  Src Port: 80
  Dst Port: 43484
HTTP Message
  HTTP/1.1 200 OK..Date: Sun, 05 Jul 2026 07:48:04 GMT..Content-Type: text/html..Transfer-Encoding: ch

===== TCP Packet =====
Ethernet Header
  Src MAC: 00:15:5d:59:d8:3d
  Dst MAC: 00:15:5d:18:e8:6c
IP Header
  Src IP: 172.66.147.243
```

9. 배우고 이해한 점 (결론)

이번 과제에서는 PCAP API를 활용하여 패킷 정보를 출력하는 프로그램을 작성하고, 캡처된 패킷을 직접 살펴보는 실습을 하였다. 이 과정을 통해 다음과 같은 점들을 이해할 수 있었다.

- 패킷은 계층 구조로 되어 있다. Ethernet → IP → TCP → 데이터 순으로 겹겹이 포장되어 있으며, 분석은 겹포장부터 한 겹씩 벗겨안으로 들어가는 과정이라는 것을 알 수 있었다.

- 고정값 대신 길이 필드를 사용해야 한다. IP-TCP 헤더는 옵션에 따라 크기가 변하므로, `iph_ihl * 4` 처럼 헤더 안의 길이 정보로 실제 크기를 계산해야 다음 헤더 위치를 정확히 찾을 수 있다는 점이 인상 깊었다.
- 바이트 순서(Endian)의 중요성을 알게 되었다. 네트워크와 PC의 숫자 저장 방식이 달라 포트·타입 값은 `ntohs()` 로 변환해야 올바른 값이 나온다는 것을 이해하였다.
- 필터링으로 원하는 패킷만 처리하는 법을 익혔다. `iph_protocol == 6` 검사와 `"tcp"` 필터로 UDP를 걸러내어 과제 조건을 충족하였다.
- HTTP 메시지는 헤더를 모두 벗겨낸 나머지임을 이해하였다. 전체 길이에서 헤더 길이를 빼는 계산이 애플리케이션 계층 데이터를 정확히 추출하는 열쇠였다.
- 이번 과제를 해결하면서 네트워크 계층구조, tcp/ip 프로토콜, curl, wsl 등의 사용 방법과 사용 이유를 면밀하게 공부해 보았다.
- Tcp/ip 프로토콜이 동작하는 방식을 이론적으로만 알고 있었는데, 이번 실습을 하며 각 계층에서 어떤정보가 이동되고, 어떻게 이해해야 하는지를 알 수 있는시간이었다.

특히 이번 실습은 단순히 코드를 작성하는 데 그치지 않고, **TCP/IP 프로토콜의 구조** 를 헤더 단위로 자세하게 이해할 수 있는 좋은 기회가 되었다. 또한 패킷 캡처가 보안에서 방어(침입 탐지)와 공격(정보 탈취) 양쪽에 어떻게 쓰이는지를 알게 되면서, 암호화가 왜 필수적인지도 근본적으로 이해할 수 있는 계기가 되었다.